

Fachbereich
Ingenieurwissenschaften

Studienbereich
Informationstechnologie
und Elektrotechnik

Studiengang
Medientechnik

Kubernetes und Terraform in der Google Cloud Plattform

Projektarbeit SS 2026
An der Hochschule RheinMain
Alina Scheiermann

Betreuer: Prof. Dr.-Ing. Bernhard Gross

Inhaltsverzeichnis

Einleitung	2
1.1 Einleitung.....	2
1.2 Zielsetzung und Anforderungen.....	3
2 Die Google Cloud Platform (GCP)	4
2.1 Compute Engine	4
2.2 Google Kubernetes Engine (GKE).....	4
2.3 Cloud Shell	5
2.4 Kostenkontrolle	6
3 Technische Grundlagen	6
3.1 Containerisierung mit Docker	6
3.2 Kubernetes	7
3.2.1 Kubernetes-Cluster	7
3.2.2 Node	8
3.2.3 Pod.....	8
3.2.4 Deployment	8
3.3 Terraform	9
3.3.1 Provider	9
3.3.2 Ressourcen	9
4 Durchführung.....	10
4.1 Überblick über die Demonstrationsszenarien	10
5 Manuelle Bereitstellung von Kubernetes mit MicroK8s.....	10
5.1 Erstellung der Compute-Engine-Instanz	11
5.2 SSH-Verbindung zur VM	11
5.3 Vorbereitung der Ubuntu-VM	12
5.4 Installation von MicroK8s	13
5.5 Konfiguration der Benutzerrechte.....	13
5.6 Demonstration der Kubernetes-Orchestrierung	14
5.7 Demonstration der Self-Healing-Funktion von Kubernetes	15
6 Automatische Bereitstellung mit Terraform.....	17
6.1 Vorbereitung der Terraform-Umgebung	17

6.2 Installation von Terraform.....	18
6.3 Aufbau der Terraform-Konfigurationsdateien	19
6.4 Initialisierung und Validierung der Terraform-Konfiguration.....	22
6.5 Terraform Plan.....	23
6.6 Terraform Apply	24
6.7 Überprüfung des GKE-Clusters	24
6.8 Verbindung zum Cluster	26
6.9 Bereitstellung der Testanwendung	26
6.10 Skalierung der Anwendung	26
6.11 Skalierung des Kubernetes-Clusters.....	27
6.12 Rückbau der Infrastruktur.....	27
7 Vergleich der manuellen Bereitstellung mit der automatisierten Bereitstellung durch Terraform	28
8 Fazit	30
Quellenverzeichnis.....	32
Abbildungsverzeichnis.....	33

Einleitung

1.1 Einleitung

Cloud Computing hat sich in den vergangenen Jahren zu einer der wichtigsten Technologien für die Bereitstellung moderner IT-Infrastrukturen entwickelt. Unternehmen und Organisationen nutzen Cloud-Plattformen zunehmend, um Anwendungen flexibel, skalierbar und kosteneffizient bereitzustellen. Insbesondere die Containerisierung von Anwendungen hat dabei erheblich an Bedeutung gewonnen, da sie eine standardisierte und portable Ausführung von Software ermöglicht.

Mit der steigenden Verbreitung von Containern wächst gleichzeitig der Bedarf an Werkzeugen, die eine große Anzahl solcher Anwendungen verwalten können. Kubernetes hat sich in diesem Bereich als De-facto-Standard für die Orchestrierung containerisierter Anwendungen etabliert. Die Plattform ermöglicht unter anderem die automatisierte Bereitstellung, Überwachung und Skalierung von Anwendungen und wird heute von zahlreichen Unternehmen produktiv eingesetzt.

Parallel dazu gewinnt die automatisierte Verwaltung von Infrastruktur zunehmend an Bedeutung. Während Cloud-Ressourcen früher häufig manuell eingerichtet wurden, kommen heute vermehrt sogenannte Infrastructure-as-Code-Ansätze zum Einsatz. Dabei wird die Infrastruktur nicht mehr über grafische Oberflächen konfiguriert, sondern mithilfe von Konfigurationsdateien beschrieben. Dies erhöht die Nachvollziehbarkeit, Wiederholbarkeit und Wartbarkeit komplexer Cloud-Umgebungen erheblich.

Vor diesem Hintergrund stellen Kubernetes und Terraform zwei zentrale Technologien moderner Cloud-Infrastrukturen dar. Die Kombination aus Container-Orchestrierung und automatisierter Infrastrukturverwaltung ermöglicht den Aufbau skalierbarer und reproduzierbarer Systeme, die den Anforderungen aktueller IT-Umgebungen gerecht werden [1-3].

1.2 Zielsetzung und Anforderungen

Ziel dieser Projektarbeit ist die praktische Untersuchung von Kubernetes und Terraform innerhalb der Google Cloud Platform (GCP). Grundlage bildet die Aufgabenstellung, die die manuelle Installation einer Kubernetes-Umgebung, die automatische Bereitstellung eines Kubernetes-Clusters mithilfe von Terraform sowie die Demonstration der Skalierbarkeit eines Kubernetes-Clusters vorsieht.

Zur Umsetzung dieser Anforderungen wird zunächst eine Kubernetes-Umgebung manuell innerhalb der Google Cloud Platform eingerichtet. Hierfür wird eine virtuelle Maschine auf Basis der Google Compute Engine verwendet, auf der Kubernetes in Form von

MicroK8s installiert und konfiguriert wird. Anschließend wird eine vergleichbare Kubernetes-Infrastruktur automatisiert mit Terraform bereitgestellt. Dabei kommen die Mechanismen des Infrastructure-as-Code-Ansatzes zum Einsatz, um die benötigten Cloud-Ressourcen reproduzierbar und nachvollziehbar zu erzeugen.

Die Funktionsfähigkeit der bereitgestellten Umgebungen wird durch die Bereitstellung einer NGINX-Anwendung überprüft. Anschließend wird die Skalierung der Anwendung demonstriert, um die Fähigkeit von Kubernetes zur Verwaltung mehrerer Instanzen einer Anwendung praktisch nachzuweisen.

Ein weiterer Schwerpunkt der Arbeit liegt auf der ausführlichen Dokumentation der verwendeten Technologien, Cloud-Ressourcen und Konfigurationsdateien. Die Dokumentation soll nicht nur die durchgeführten Experimente nachvollziehbar darstellen, sondern gleichzeitig als technische Grundlage für zukünftige Projekte im Bereich Kubernetes, Terraform und Cloud Computing dienen.

Für die praktische Umsetzung werden verschiedene Dienste der Google Cloud Platform eingesetzt. Die hierfür relevanten Technologien und Grundlagen werden im folgenden Kapitel erläutert.

2 Die Google Cloud Platform (GCP)

Cloud Computing beschreibt die Bereitstellung von IT-Ressourcen wie Rechenleistung, Speicherplatz oder Netzwerkdiensten über das Internet. Anstatt eigene Server zu betreiben, können Unternehmen und Entwickler diese Ressourcen bedarfsgerecht bei Cloud-Anbietern nutzen. Dadurch lassen sich Anwendungen flexibel bereitstellen und je nach Auslastung erweitern oder reduzieren.

Für die Durchführung dieser Projektarbeit wurde die Google Cloud Platform (GCP) verwendet. Die GCP stellt eine Vielzahl von Cloud-Diensten zur Verfügung, die für den Aufbau, die Verwaltung und die Automatisierung moderner IT-Infrastrukturen genutzt werden können. Dazu gehören unter anderem virtuelle Maschinen, Container-Plattformen, Speicherdienste sowie Werkzeuge zur Infrastrukturautomatisierung.

Im Rahmen dieser Arbeit dient die Google Cloud Platform als technische Grundlage für die manuelle Installation einer Kubernetes-Umgebung sowie für die automatisierte Bereitstellung eines Kubernetes-Clusters mit Terraform. Die folgenden Abschnitte beschreiben die wichtigsten hierfür eingesetzten Dienste und deren Aufgaben innerhalb der Projektumgebung.

2.1 Compute Engine

Google Compute Engine ist der Dienst der Google Cloud Platform zur Bereitstellung virtueller Maschinen. Eine virtuelle Maschine stellt einen vollständig konfigurierbaren Rechner

innerhalb der Cloud dar und ermöglicht die Installation beliebiger Betriebssysteme und Anwendungen.

Für die manuelle Kubernetes-Installation wurde eine Ubuntu-basierte virtuelle Maschine in der Region europe-west3 (Frankfurt) erstellt. Auf dieser Maschine erfolgte die Installation und Konfiguration von MicroK8s als Kubernetes-Umgebung. Die Compute Engine bietet hierbei den Vorteil, dass sämtliche Installations- und Konfigurationsschritte eigenständig durchgeführt werden können und somit ein tiefer Einblick in die Funktionsweise eines Kubernetes-Clusters ermöglicht wird.

Darüber hinaus erlaubt die Compute Engine die flexible Anpassung von Rechenleistung, Arbeitsspeicher und Speicherplatz. Dadurch eignet sie sich sowohl für Testumgebungen als auch für produktive Anwendungen unterschiedlicher Größenordnungen [5].

2.2 Google Kubernetes Engine (GKE)

Die Google Kubernetes Engine (GKE) ist ein von Google bereitgestellter Dienst zur Verwaltung von Kubernetes-Clustern innerhalb der Google Cloud Platform. Kubernetes ist eine Plattform zur Orchestrierung containerisierter Anwendungen und ermöglicht die automatisierte Bereitstellung, Verwaltung und Skalierung von Containern.

Im Gegensatz zu einer selbst installierten Kubernetes-Umgebung übernimmt Google bei der Google Kubernetes Engine zahlreiche Verwaltungsaufgaben. Dazu gehören unter anderem die Bereitstellung der Kubernetes-Kontrollebene, Software-Updates, Sicherheitsaktualisierungen sowie die Überwachung der Cluster-Infrastruktur. Dadurch reduziert sich der administrative Aufwand erheblich, sodass sich Entwickler stärker auf die eigentlichen Anwendungen konzentrieren können.

Ein Kubernetes-Cluster in der Google Kubernetes Engine besteht aus einer Kontrollebene sowie einem oder mehreren Worker-Nodes. Auf diesen Worker-Nodes werden die containerisierten Anwendungen in Form von Pods ausgeführt. Die Verwaltung und Verteilung der Pods erfolgt automatisch durch Kubernetes.

Im Rahmen dieser Projektarbeit wird die Google Kubernetes Engine für die automatisierte Bereitstellung eines Kubernetes-Clusters mithilfe von Terraform eingesetzt. Hierdurch kann untersucht werden, wie sich Cloud-Infrastrukturen reproduzierbar und automatisiert erstellen lassen. Gleichzeitig bietet die Google Kubernetes Engine eine geeignete Plattform, um die Bereitstellung und Skalierung einer containerisierten Anwendung praktisch zu demonstrieren [6].

2.3 Cloud Shell

Die Google Cloud Shell ist eine browserbasierte Linux-Umgebung, die direkt innerhalb der Google Cloud Platform zur Verfügung gestellt wird. Sie ermöglicht den Zugriff auf Cloud-Ressourcen, ohne dass zusätzliche Software auf dem lokalen Rechner installiert werden muss.

Die Umgebung enthält bereits zahlreiche Werkzeuge, die für die Verwaltung von Cloud-Infrastrukturen benötigt werden. Dazu gehören unter anderem die Google Cloud CLI (gcloud), kubectl zur Verwaltung von Kubernetes-Clustern sowie Terraform zur Infrastrukturautomatisierung. Dadurch können viele Verwaltungsaufgaben unmittelbar über den Webbrowser durchgeführt werden.

Ein weiterer Vorteil der Cloud Shell besteht darin, dass die Authentifizierung gegenüber der Google Cloud Platform bereits vorkonfiguriert ist. Benutzer können daher direkt auf ihre Projekte und Ressourcen zugreifen, ohne zusätzliche Zugangsdaten hinterlegen zu müssen.

Im Rahmen dieser Projektarbeit wurde die Cloud Shell insbesondere für die Arbeit mit Terraform genutzt. Die Erstellung, Validierung und Ausführung der Terraform-Konfigurationen erfolgte vollständig innerhalb der bereitgestellten Cloud-Umgebung. Darüber hinaus wurde die Cloud Shell verwendet, um Verbindungen zu Kubernetes-Clustern herzustellen und deren Zustand mithilfe von kubectl zu überprüfen [7].

2.4 Kostenkontrolle

Für die Durchführung der Projektarbeit stellte die Hochschule ein Google-Cloud-Guthaben in Höhe von 50,00 US-Dollar zur Verfügung. Zu Beginn der praktischen Arbeit stand das vollständige Guthaben zur Verfügung, wodurch sämtliche geplanten Experimente innerhalb des bereitgestellten Budgets durchgeführt werden konnten.

Da die Google Cloud Platform nach dem Pay-as-you-go-Prinzip arbeitet, entstehen Kosten ausschließlich für tatsächlich genutzte Cloud-Ressourcen. Aus diesem Grund wurde während der gesamten Projektarbeit darauf geachtet, nicht mehr benötigte Ressourcen nach Abschluss der jeweiligen Experimente konsequent zu stoppen beziehungsweise vollständig zu entfernen. Insbesondere wurden mit Terraform bereitgestellte Ressourcen nach Abschluss der Tests mithilfe des Befehls `terraform destroy` wieder gelöscht, um unnötige Kosten zu vermeiden [8].

Guthabenname ↑	Status ↑	Prozent verbleibend	Restwert	Ursprünglicher Wert	Typ
Computer & Media Networking II	✓ Verfügbar	100 %	50,00 \$	50,00 \$	Einmal

Abbildung 1: Bereitgestelltes Google-Cloud-Guthaben zu Beginn der Projektarbeit

3 Technische Grundlagen

3.1 Containerisierung mit Docker

Die Containerisierung beschreibt ein Verfahren, bei dem Anwendungen gemeinsam mit den benötigten Abhängigkeiten in einer standardisierten und isolierten Umgebung ausgeführt werden. Ein Container enthält dabei die Anwendung selbst sowie die zur Ausführung erforderlichen Bestandteile wie Bibliotheken, Laufzeitumgebungen und Konfigurationsdateien. Dadurch kann eine Anwendung zuverlässiger zwischen verschiedenen Systemumgebungen übertragen und ausgeführt werden.

Docker ist eine Plattform zur Erstellung, Bereitstellung und Ausführung containerisierter Anwendungen. Im Unterschied zu klassischen virtuellen Maschinen benötigen Container kein eigenes vollständiges Betriebssystem, sondern nutzen den Kernel des Hostsystems gemeinsam. Dadurch können Container ressourcenschonender betrieben und schneller gestartet werden als virtuelle Maschinen.

Für Kubernetes ist die Containerisierung eine zentrale Grundlage, da Kubernetes für die Verwaltung containerisierter Anwendungen entwickelt wurde. Im Rahmen dieser Projektarbeit wurde zur Funktionsprüfung eine NGINX-Anwendung als Container innerhalb eines Kubernetes-Clusters bereitgestellt und anschließend skaliert [1].

3.2 Kubernetes

Kubernetes ist eine Open-Source-Plattform zur Orchestrierung containerisierter Anwendungen. Die Plattform wurde ursprünglich von Google entwickelt und wird heute von der Cloud Native Computing Foundation (CNCF) weitergeführt. Kubernetes ermöglicht die automatisierte Bereitstellung, Verwaltung und Skalierung von Anwendungen, die in Containern ausgeführt werden.

Die Hauptaufgabe von Kubernetes besteht darin, den gewünschten Zustand einer Anwendung aufrechtzuerhalten. Hierzu überwacht das System kontinuierlich die laufenden Container und reagiert automatisch auf Veränderungen. Fällt beispielsweise ein Container aus, kann Kubernetes diesen automatisch neu starten oder ersetzen. Darüber hinaus unterstützt Kubernetes die Verteilung von Anwendungen auf mehrere Systeme sowie die horizontale Skalierung durch zusätzliche Instanzen einer Anwendung.

Durch diese Eigenschaften hat sich Kubernetes als Standardlösung für den Betrieb containerisierter Anwendungen in Cloud-Umgebungen etabliert. Für das Verständnis der praktischen Umsetzung sind insbesondere die Konzepte Cluster, Node, Pod, Deployment und Service von Bedeutung [2].

3.2.1 Kubernetes-Cluster

Ein Kubernetes-Cluster bildet die grundlegende Verwaltungseinheit einer Kubernetes-Umgebung. Ein Cluster besteht aus einer Kontrollebene (Control Plane) sowie einem oder mehreren Worker-Nodes. Die Kontrollebene verwaltet den Zustand des Clusters und koordiniert die Ausführung der Anwendungen, während die Worker-Nodes die eigentlichen Container ausführen.

Kubernetes verfolgt dabei das Prinzip des gewünschten Zustands (Desired State). Der Benutzer definiert, in welchem Zustand sich die Anwendungen befinden sollen. Kubernetes überwacht anschließend kontinuierlich den Cluster und führt automatisch Maßnahmen durch, um diesen Zustand aufrechtzuerhalten.

Im Rahmen dieser Projektarbeit wurden sowohl ein manuell bereitgestellter Kubernetes-Cluster auf Basis von MicroK8s als auch ein automatisiert mit Terraform erzeugter Cluster innerhalb der Google Kubernetes Engine verwendet [2].

3.2.2 Node

Ein Node stellt einen einzelnen Rechner innerhalb eines Kubernetes-Clusters dar. Dabei kann es sich um eine physische oder virtuelle Maschine handeln. Auf den Nodes werden die Container der Anwendungen ausgeführt, die von Kubernetes verwaltet werden.

Jeder Node enthält die für den Betrieb von Containern erforderlichen Komponenten. Kubernetes verteilt die Anwendungen automatisch auf die verfügbaren Nodes und überwacht deren Zustand. Fällt ein Node aus, kann Kubernetes die betroffenen Anwendungen auf andere verfügbare Nodes verschieben [2].

3.2.3 Pod

Ein Pod ist die kleinste bereitstellbare Einheit innerhalb eines Kubernetes-Clusters. Ein Pod enthält einen oder mehrere Container, die gemeinsam auf demselben Node ausgeführt werden. Die Container innerhalb eines Pods teilen sich Netzwerkressourcen sowie Speicherressourcen und können dadurch eng zusammenarbeiten.

Kubernetes verwaltet Anwendungen nicht direkt auf Container-Ebene, sondern auf Pod-Ebene. Wird ein Pod erstellt, gestartet oder beendet, betrifft dies alle darin enthaltenen Container. In den meisten Anwendungsfällen enthält ein Pod jedoch lediglich einen einzelnen Container.

Pods sind grundsätzlich kurzlebig. Fällt ein Pod aus oder wird gelöscht, kann Kubernetes automatisch einen neuen Pod erzeugen, um den gewünschten Zustand der Anwendung wiederherzustellen. Dadurch wird die Verfügbarkeit der Anwendung erhöht und der Verwaltungsaufwand reduziert [2].

3.2.4 Deployment

Ein Deployment ist ein Kubernetes-Objekt zur Verwaltung von Pods. Es beschreibt den gewünschten Zustand einer Anwendung und legt unter anderem fest, welches Container-Image verwendet werden soll und wie viele Instanzen der Anwendung ausgeführt werden sollen.

Kubernetes überwacht kontinuierlich den tatsächlichen Zustand der Anwendung und vergleicht diesen mit dem im Deployment definierten Soll-Zustand. Werden Pods gelöscht oder fallen aus, erstellt Kubernetes automatisch neue Pods, um die vorgegebene Anzahl von Instanzen aufrechtzuerhalten.

Darüber hinaus ermöglichen Deployments die einfache Skalierung von Anwendungen. Hierbei kann die Anzahl der Pod-Replikat erhöht oder reduziert werden, ohne dass die Anwendung neu installiert werden muss. Dadurch stellen Deployments ein zentrales Werkzeug für die Verwaltung containerisierter Anwendungen innerhalb eines Kubernetes-Clusters dar [2].

3.3 Terraform

Terraform ist ein Infrastructure-as-Code-Werkzeug zur automatisierten Bereitstellung und Verwaltung von IT-Infrastrukturen. Es ermöglicht die deklarative Definition von Cloud-Ressourcen in Konfigurationsdateien.

Die gewünschte Infrastruktur wird in diesen Konfigurationsdateien beschrieben und anschließend von Terraform ausgewertet und umgesetzt. Hierdurch können Cloud-Ressourcen reproduzierbar erstellt, geändert und entfernt werden. Änderungen an der Infrastruktur werden dabei nachvollziehbar dokumentiert und können bei Bedarf erneut ausgeführt werden [3].

3.3.1 Provider

Terraform verwendet sogenannte Provider, um mit externen Plattformen und Diensten zu kommunizieren. Ein Provider stellt die Verbindung zwischen Terraform und der jeweiligen Infrastruktur her und ermöglicht die Verwaltung der dort verfügbaren Ressourcen.

Für die automatische Bereitstellung der Infrastruktur wurde in dieser Projektarbeit der Google-Provider verwendet. Dieser ermöglicht Terraform den Zugriff auf Ressourcen der Google Cloud Platform und stellt die benötigten Funktionen für die Erstellung und Verwaltung eines Kubernetes-Clusters bereit [3, 10].

3.3.2 Ressourcen

Ressourcen bilden die zentralen Bausteine einer Terraform-Konfiguration. Jede Ressource beschreibt ein konkretes Objekt innerhalb der Zielplattform, beispielsweise eine virtuelle Maschine, ein Netzwerk oder einen Kubernetes-Cluster.

Die Eigenschaften einer Ressource werden innerhalb der Terraform-Konfiguration definiert. Terraform vergleicht die gewünschte Konfiguration mit dem aktuellen Zustand der Infrastruktur und führt die erforderlichen Änderungen automatisch durch [3, 10].

4 Durchführung

Die praktische Durchführung dient dazu, die in den vorherigen Kapiteln beschriebenen Technologien anhand eines zusammenhängenden Anwendungsszenarios praktisch zu untersuchen. Dabei soll gezeigt werden, wie sich eine Kubernetes-Umgebung sowohl manuell als auch automatisiert bereitstellen lässt und welche Möglichkeiten Kubernetes für die Verwaltung, Skalierung und Ausfallsicherheit containerisierter Anwendungen bietet. Die einzelnen Demonstrationsszenarien bauen aufeinander auf und werden in den folgenden Kapiteln schrittweise umgesetzt und dokumentiert.

4.1 Überblick über die Demonstrationsszenarien

Kubernetes bietet verschiedene Möglichkeiten, die Funktionen einer Container-Orchestrierungsplattform anhand praktischer Demonstrationsszenarien zu veranschaulichen. Zu den häufigsten Demonstrationen zählen die Bereitstellung containerisierter Anwendungen, die Skalierung von Pods, die automatische Wiederherstellung ausgefallener Container (Self-Healing), Rolling Updates sowie die automatische Skalierung mithilfe des Horizontal Pod Autoscalers (HPA). Die einzelnen Szenarien verdeutlichen unterschiedliche Eigenschaften der Kubernetes-Orchestrierung und werden je nach Zielsetzung eingesetzt.

Zunächst erfolgt die manuelle Bereitstellung einer Kubernetes-Umgebung mit MicroK8s auf einer virtuellen Maschine. Anschließend wird ein Kubernetes-Cluster mithilfe von Terraform automatisiert in der Google Kubernetes Engine (GKE) bereitgestellt. Die Funktionsfähigkeit der bereitgestellten Umgebung wird durch die Bereitstellung einer containerisierten NGINX-Anwendung überprüft. Darauf aufbauend werden die Skalierung der Anwendung sowie die Self-Healing-Funktion von Kubernetes demonstriert.

Die praktische Umsetzung verfolgt das Ziel, die Unterschiede zwischen der manuellen und der automatisierten Bereitstellung einer Kubernetes-Umgebung aufzuzeigen und gleichzeitig zentrale Funktionen der Kubernetes-Orchestrierung praxisnah zu demonstrieren. Die einzelnen Experimente bauen aufeinander auf und werden in den folgenden Kapiteln schrittweise beschrieben und dokumentiert.

5 Manuelle Bereitstellung von Kubernetes mit MicroK8s

5.1 Erstellung der Compute-Engine-Instanz

Für die manuelle Installation von Kubernetes wurde zunächst eine virtuelle Maschine in der Google Compute Engine erstellt. Als Standort wurde die Region europe-west3 (Frankfurt) mit der Zone europe-west3-b gewählt.

Für die virtuelle Maschine wurde ein Instanztyp der E2-Serie verwendet. Als konkrete Konfiguration wurde der Maschinentyp e2-small mit 2 vCPUs und 2 GB Arbeitsspeicher ausgewählt. Da die Umgebung ausschließlich für Test- und Demonstrationszwecke genutzt wurde, wurde ein kostengünstiger Maschinentyp gewählt.

Als Betriebssystem kam Ubuntu 24.04 LTS Minimal zum Einsatz. Für das Bootlaufwerk wurde ein nichtflüchtiger Standardspeicher mit einer Kapazität von 20 GB konfiguriert.

Nach Abschluss der Konfiguration wurde die virtuelle Maschine erstellt und für die Installation von MicroK8s vorbereitet.

The screenshot shows the configuration page for a new Compute Engine instance. The instance name is 'microk8s-manual'. The region is set to 'europe-west3 (Frankfurt)' and the zone is 'europe-west3-b'. The machine type is 'E2', specifically 'e2-small', which is described as 'Kostengünstiges tägliches Computing' with 0.25 - 32 vCPUs and 1-128 GB of memory. The image is 'Ubuntu 24.04 LTS Minimal'. The disk type is 'Neuer abgestimmter nichtflüchtiger Speicher' with a size of 20 GB. The snapshot schedule is 'default-schedule-1' and the license type is 'Kostenlos'. There is an 'Ändern' (Change) button at the bottom.

Name	microk8s-manual
Typ	Neuer abgestimmter nichtflüchtiger Speicher
Größe	20 GB
Zeitplan für Snapshots	default-schedule-1
Lizenztyp	Kostenlos
Image	Ubuntu 24.04 LTS Minimal

Abbildung 2: Konfiguration der Compute-Engine-Instanz

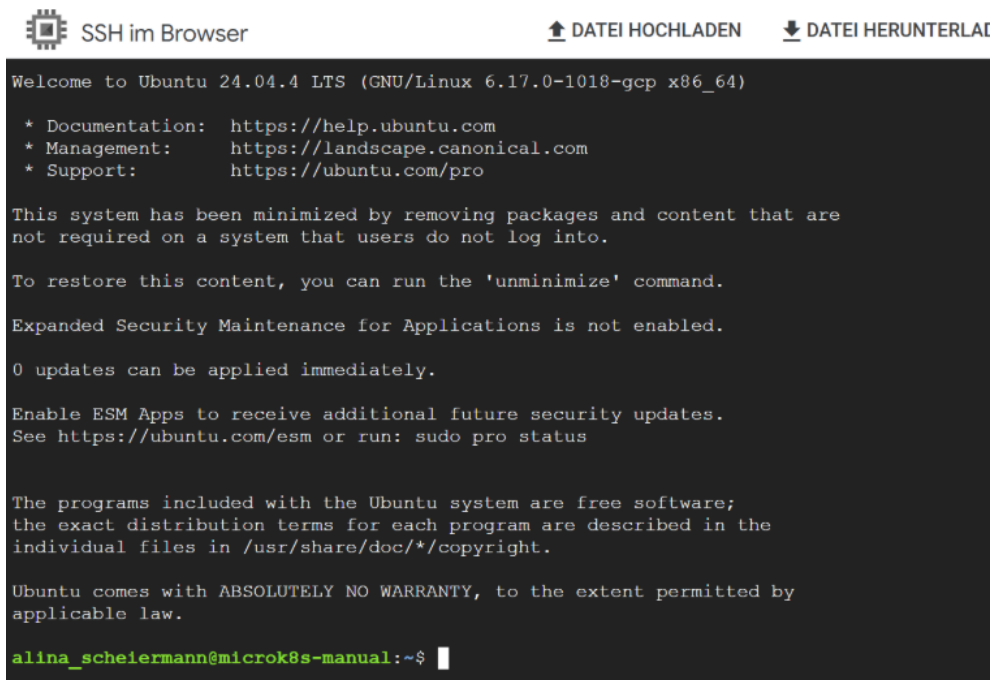
The screenshot shows the 'VM-Instanzen' (VM Instances) page. It contains a table with one instance listed: 'microk8s-manual' in the 'europe-west3-b' zone. The instance is running on the 'e2-small' machine type. The internal IP is 10.156.0.37 and the external IP is 34.141.91.27. The instance is connected via SSH.

Status	Name	Zone	Empfehlungen	Verwendet von	Interne IP	Externe IP-Adresse	Verbinden
✓	microk8s-manual	europe-west3-b			10.156.0.37 (nic0)	34.141.91.27 (nic0)	SSH

Abbildung 3: Erstellte VM-Instanz

5.2 SSH-Verbindung zur VM

Nach der Erstellung der virtuellen Maschine wurde über die SSH-Funktion der Google Cloud Platform eine Verbindung zur Ubuntu-Umgebung hergestellt. Die erfolgreiche Anmeldung zeigt die betriebsbereite Bereitstellung der Instanz und bildet die Grundlage für die weitere Installation von MicroK8s [5].



```
SSH im Browser  ⬆️ DATEI HOCHLADEN  ⬇️ DATEI HERUNTERLADEN

Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1018-gcp x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

alina_scheiermann@microk8s-manual:~$
```

Abbildung 4: SSH-Verbindung zur Ubuntu-VM

5.3 Vorbereitung der Ubuntu-VM

Vor der Installation von MicroK8s wurde das Betriebssystem aktualisiert, um sicherzustellen, dass alle installierten Pakete auf dem aktuellen Stand sind. Anschließend wurde die vorhandene Snap-Installation überprüft, da MicroK8s über den Snap-Paketmanager bereitgestellt wird [4]. Die Ausgabe von `snap version` bestätigte die korrekte Installation und Funktionsfähigkeit von Snap (Abbildung 5).

```
sudo apt update
```

```
sudo apt upgrade -y
```

```
alina_scheiermann@microk8s-manual:~$ snap version
snap          2.75.2+ubuntu24.04
snapd         2.75.2+ubuntu24.04
series        16
ubuntu        24.04
kernel        6.17.0-1018-gcp
architecture  amd64
alina_scheiermann@microk8s-manual:~$
```

Abbildung 5: Überprüfung der Snap-Installation

5.4 Installation von MicroK8s

Die Installation von MicroK8s erfolgte über den Snap-Paketmanager mit dem Befehl `sudo snap install microk8s --classic`. Die Option `--classic` ermöglicht den erweiterten Zugriff auf Systemressourcen, der für den Betrieb von Kubernetes erforderlich ist. Nach erfolgreicher Installation stand die Kubernetes-Umgebung für die weitere Konfiguration zur Verfügung [4].

```
alina_scheiermann@microk8s-manual:~$ sudo snap install microk8s --classic
microk8s (1.35/stable) v1.35.0 from Canonical✓ installed
```

Abbildung 6: Installation von MicroK8s

Nach der Installation von MicroK8s konnten aufgrund fehlender Benutzerberechtigungen zunächst keine Statusinformationen des Clusters abgerufen werden. Die von der MicroK8s-Dokumentation empfohlenen Benutzerrechte wurden daraufhin gesetzt, um den Zugriff ohne Root-Rechte zu ermöglichen [4].

```
alina_scheiermann@microk8s-manual:~$ microk8s status --wait-ready
Insufficient permissions to access MicroK8s.
You can either try again with sudo or add the user alina_scheiermann to the 'microk8s' group:

sudo usermod -a -G microk8s alina_scheiermann
sudo chown -R alina_scheiermann ~/.kube
```

Abbildung 7: Fehlende Benutzerberechtigungen

5.5 Konfiguration der Benutzerrechte

Nach der Installation von MicroK8s konnte zunächst nicht auf den Kubernetes-Cluster zugegriffen werden. Der Aufruf des Befehls `microk8s status --wait-ready` führte zu einer Fehlermeldung, da dem verwendeten Benutzer die erforderlichen Berechtigungen fehlten (Abbildung 7).

Zur Behebung des Problems wurden die von MicroK8s empfohlenen Benutzerrechte gesetzt. Anschließend wurde die Benutzergruppe neu geladen und der Status des Clusters erneut überprüft.

Die erfolgreiche Ausgabe von `microk8s status --wait-ready` belegte die betriebsbereite Installation von MicroK8s sowie die erfolgreiche Konfiguration der Benutzerrechte (Abbildung 8).

```
alina_scheiermann@microk8s-manual:~$ microk8s status --wait-ready
microk8s is running
high-availability: no
datastore master nodes: 127.0.0.1:19001
datastore standby nodes: none
addons:
```

Abbildung 8: Initialisierung von MicroK8s

Nach der Initialisierung wurde der Zustand des Clusters mit dem Befehl `microk8s kubectl get nodes -o wide` überprüft. Der Node `microk8s-manual` befindet sich im Status *Ready*, wodurch die erfolgreiche Bereitstellung des Kubernetes-Clusters bestätigt wird (Abbildung 9).

```
alina_scheiermann@microk8s-manual:~$ microk8s kubectl get nodes -o wide
NAME                STATUS  ROLES  AGE  VERSION  INTERNAL-IP  EXTERNAL-IP  OS-IMAGE
microk8s-manual     Ready   <none>  14m  v1.35.0  10.156.0.37  <none>       Ubuntu 24.04.4 LTS
6.17.0-1018-gcp     containerd://2.1.3
```

Abbildung 9: Kubernetes Node nach erfolgreicher Initialisierung

5.6 Demonstration der Kubernetes-Orchestrierung

Zur Überprüfung der Funktionsfähigkeit des Kubernetes-Clusters wurde eine NGINX-Anwendung als Testanwendung bereitgestellt. Hierfür wurde ein Deployment auf Basis des offiziellen NGINX-Container-Images erstellt (Abbildung 10) [2].

```
alina_scheiermann@microk8s-manual:~$ microk8s kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
alina_scheiermann@microk8s-manual:~$ microk8s kubectl get deployments
NAME    READY  UP-TO-DATE  AVAILABLE  AGE
nginx   1/1    1            1           34s
alina_scheiermann@microk8s-manual:~$ microk8s kubectl get pods
NAME                READY  STATUS  RESTARTS  AGE
nginx-56c45fd5ff-7mrp  1/1    Running  0          55s
alina_scheiermann@microk8s-manual:~$
```

Abbildung 10: Erstellung NGINX Deployments

Die anschließende Kontrolle zeigte ein erfolgreich erstelltes Deployment sowie einen laufenden Pod im Status *Running* (Abbildung 10).

Zur Demonstration der Skalierbarkeit von Kubernetes wurde die Anzahl der Replikate des NGINX-Deployments von einem auf drei erhöht. Kubernetes erzeugte daraufhin automatisch zwei zusätzliche Pods. Die Überprüfung mit `microk8s kubectl get pods` ergab, dass alle drei Pods erfolgreich gestartet wurden und sich im Status *Running* befinden (Abbildung 11) [2].


```

alina_scheiermann@microk8s-manual:~$ microk8s kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled
alina_scheiermann@microk8s-manual:~$ microk8s kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
nginx-56c45fd5ff-7mrwp             1/1     Running   0           22m
nginx-56c45fd5ff-jcx6w             1/1     Running   0           20s
nginx-56c45fd5ff-tbjhh             1/1     Running   0           20s

```

Abbildung 11: Skalierung auf drei Pods

```

NAME                                READY   STATUS    RESTARTS   AGE
pod/nginx-56c45fd5ff-7mrwp         1/1     Running   0           57m
pod/nginx-56c45fd5ff-jcx6w         1/1     Running   0           35m
pod/nginx-56c45fd5ff-tbjhh         1/1     Running   0           35m

NAME                                TYPE          CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/kubernetes                 ClusterIP      10.152.183.1    <none>        443/TCP    76m
service/nginx                      ClusterIP      10.152.183.234  <none>        80/TCP     77s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx              3/3     3             3           57m

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/nginx-56c45fd5ff    3         3         3       57m

```

Abbildung 12: NGINX-Service nach erfolgreicher Skalierung

Nach der Skalierung wurden die Kubernetes-Ressourcen überprüft. Die Ausgabe von `microk8s kubectl get all` zeigt das NGINX-Deployment mit drei laufenden Pods sowie den zugehörigen Service. Über den Service wurde die Cluster-IP 10.152.183.234 bereitgestellt, über die die Anwendung innerhalb des Clusters erreichbar ist [2].

Zur Überprüfung der Funktionsfähigkeit wurde die zuvor ermittelte Cluster-IP des NGINX-Services mit dem Befehl `curl 10.152.183.234` aufgerufen. Die zurückgelieferte Standard-Webseite von NGINX bestätigt die erfolgreiche Bereitstellung und Erreichbarkeit der Anwendung innerhalb des Kubernetes-Clusters.

```

alina_scheiermann@microk8s-manual:~$ curl 10.152.183.234
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, nginx is successfully installed and working.
Further configuration is required for the web server, reverse proxy,
API gateway, load balancer, content cache, or other features.</p>

<p>For online documentation and support please refer to
<a href="https://nginx.org/">nginx.org</a>.<br/>
To engage with the community please visit
<a href="https://community.nginx.org/">community.nginx.org</a>.<br/>
For enterprise grade support, professional services, additional
security features and capabilities please refer to
<a href="https://f5.com/nginx">f5.com/nginx</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>

```

Abbildung 13: Zugriff auf die NGINX-Anwendung

5.7 Demonstration der Self-Healing-Funktion von Kubernetes

Nach der erfolgreichen Bereitstellung und Skalierung der NGINX-Testanwendung wurde zusätzlich die Self-Healing-Funktion von Kubernetes demonstriert. Diese zählt zu den zentralen Eigenschaften einer Container-Orchestrierungsplattform und beschreibt die Fähigkeit, ausgefallene Pods automatisch zu erkennen und selbstständig zu ersetzen. Ziel dieses Demonstrationsszenarios war es nachzuweisen, dass Kubernetes den gewünschten Sollzustand eines Deployments kontinuierlich überwacht und nach dem Ausfall eines Pods automatisch wiederherstellt [2, 4].

Für die Demonstration der Self-Healing-Funktion wurde das NGINX-Deployment mit drei Replikaten erneut bereitgestellt. Dadurch unterscheiden sich die in diesem Abschnitt dargestellten Pod-Namen von denen des zuvor gezeigten Skalierungsszenarios. Dies ist auf die automatische Namensvergabe von Kubernetes bei jeder neuen Bereitstellung eines Deployments zurückzuführen.

Zur Simulation eines Ausfalls wurde gezielt einer der laufenden Pods mit dem Befehl `kubectl delete pod` manuell gelöscht. Dadurch wich der tatsächliche Zustand des Deployments kurzfristig vom gewünschten Sollzustand ab.

```
alina_scheiermann_student@microk8s-manual:~$ sudo microk8s kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-56c45fd5ff-45gn8	1/1	Running	0	75s
nginx-56c45fd5ff-pxr77	1/1	Running	0	14s
nginx-56c45fd5ff-zw9xf	1/1	Running	0	14s

Abbildung 14: Ausgangszustand des NGINX-Deployments mit drei laufenden Pods vor der Demonstration der Self-Healing-Funktion.

Unmittelbar nach dem Löschen des Pods erkannte Kubernetes die Abweichung vom definierten Sollzustand des Deployments. Kubernetes erzeugte automatisch einen neuen Pod und vergab diesem einen neuen eindeutigen Namen, während die Anzahl der laufenden Pods unverändert bei drei blieb. Bereits nach kurzer Zeit befanden sich wieder drei Pods im Status *Running*. Der ursprünglich gelöschte Pod wurde dabei durch einen neuen Pod mit einem anderen Namen ersetzt.

```
alina_scheiermann_student@microk8s-manual:~$ sudo microk8s kubectl delete pod nginx-56c45fd5ff-45gn8
pod "nginx-56c45fd5ff-45gn8" deleted from default namespace
alina_scheiermann_student@microk8s-manual:~$ sudo microk8s kubectl get pods -w
```

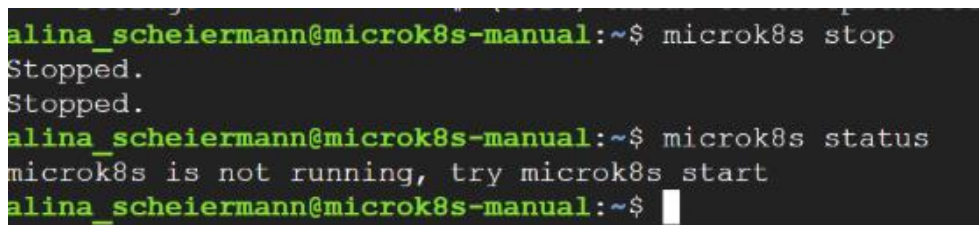
NAME	READY	STATUS	RESTARTS	AGE
nginx-56c45fd5ff-msld9	1/1	Running	0	29s
nginx-56c45fd5ff-pxr77	1/1	Running	0	5m55s
nginx-56c45fd5ff-zw9xf	1/1	Running	0	5m55s

Abbildung 15: Manuelles Löschen eines Pods und automatische Wiederherstellung des Deployments durch Kubernetes (Self-Healing).

Das Experiment verdeutlicht die Self-Healing-Funktion von Kubernetes. Kubernetes überwacht dabei nicht die einzelnen Pods, sondern den im Deployment definierten Sollzustand. Fällt ein Pod aus oder wird entfernt, sorgt der Deployment-Controller automatisch für die Wiederherstellung der gewünschten Anzahl an Replikaten. Dadurch werden ausgefallene Pods ohne manuellen Eingriff ersetzt und die Verfügbarkeit der containerisierten Anwendung sichergestellt. Dieses Verhalten verdeutlicht eine zentrale Funktion der Kubernetes-Orchestrierung und bestätigt die Fähigkeit von Kubernetes, die Verfügbarkeit und Ausfallsicherheit containerisierter Anwendungen durch die automatische Wiederherstellung des gewünschten Sollzustands sicherzustellen.

Die Demonstration zeigt praxisnah, dass Kubernetes den gewünschten Sollzustand eines Deployments kontinuierlich sicherstellt und Abweichungen automatisch korrigiert. Im Gegensatz zu einer rein manuellen Containerverwaltung erfolgt die Wiederherstellung eines ausgefallenen Pods ohne Eingriff des Administrators. Dadurch wird deutlich, weshalb Kubernetes insbesondere für den zuverlässigen Betrieb containerisierter Anwendungen in produktiven Cloud-Umgebungen geeignet ist.

Nach Abschluss der Experimente wurde der MicroK8s-Cluster kontrolliert gestoppt. Die anschließende Statusabfrage bestätigte, dass die Kubernetes-Umgebung nicht mehr aktiv ist. Durch das Stoppen nicht benötigter Ressourcen können unnötige Kosten und Ressourcennutzung vermieden werden.



```
alina_scheiermann@microk8s-manual:~$ microk8s stop
Stopped.
Stopped.
alina_scheiermann@microk8s-manual:~$ microk8s status
microk8s is not running, try microk8s start
alina_scheiermann@microk8s-manual:~$
```

Abbildung 16: Kontrolliertes Stoppen des MicroK8s-Clusters

Damit wurde die manuelle Installation und Konfiguration eines Kubernetes-Clusters auf Basis von MicroK8s erfolgreich abgeschlossen.

6 Automatische Bereitstellung mit Terraform

6.1 Vorbereitung der Terraform-Umgebung

Für die automatisierte Bereitstellung der Kubernetes-Infrastruktur wurde die Google Cloud Shell verwendet. Die Cloud Shell stellt eine browserbasierte Linux-Umgebung bereit und ermöglicht die direkte Verwaltung von Ressourcen innerhalb der Google Cloud Platform.

Zu Beginn wurde innerhalb der Cloud Shell ein separates Arbeitsverzeichnis für die Terraform-Konfiguration erstellt. Dieses diente als Ablageort für die später angelegten Konfigurationsdateien (`main.tf`, `variables.tf` und `outputs.tf`) sowie für die weitere Bearbeitung der Terraform-Umgebung.

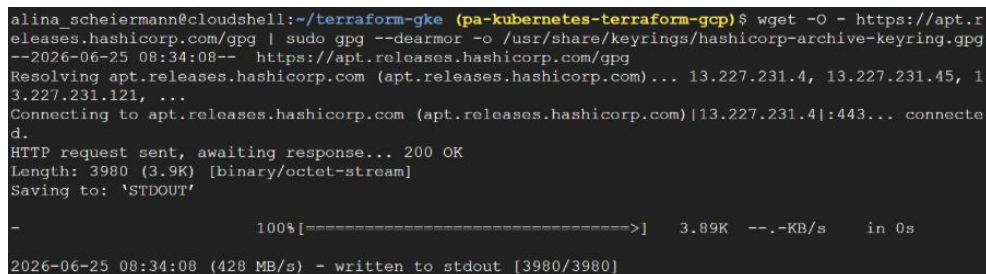
Nach Abschluss der Vorbereitungen konnte mit der Installation von Terraform und der Erstellung der Konfigurationsdateien begonnen werden [3, 7].

6.2 Installation von Terraform

Für die automatisierte Bereitstellung der Kubernetes-Infrastruktur wurde Terraform eingesetzt. Obwohl Terraform in der Google Cloud Shell bereits vorinstalliert ist, wurde die Installation über das offizielle HashiCorp-Repository durchgeführt, um den Installationsprozess entsprechend der offiziellen Dokumentation nachvollziehbar zu dokumentieren.

Der Import des offiziellen GPG-Schlüssels ermöglicht die kryptografische Überprüfung der Authentizität und Integrität der später installierten Pakete. Dadurch wird sichergestellt, dass ausschließlich vertrauenswürdige Software aus dem offiziellen HashiCorp-Repository installiert wird.

```
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
```



```
alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp)$ wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg
--2026-06-25 08:34:08-- https://apt.releases.hashicorp.com/gpg
Resolving apt.releases.hashicorp.com (apt.releases.hashicorp.com)... 13.227.231.4, 13.227.231.45, 13.227.231.121, ...
Connecting to apt.releases.hashicorp.com (apt.releases.hashicorp.com)|13.227.231.4|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3980 (3.9K) [binary/octet-stream]
Saving to: 'STDOUT'

- 100%[=====>] 3.89K --KB/s in 0s

2026-06-25 08:34:08 (428 MB/s) - written to stdout [3980/3980]
```

Abbildung 17: Import des offiziellen HashiCorp-GPG-Schlüssels.

```
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] https://apt.releases.hashicorp.com noble main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
```

Das Repository wurde entsprechend der verwendeten Ubuntu-Version (Codename noble) eingebunden.

Anschließend wurde das offizielle HashiCorp-Repository als zusätzliche Paketquelle in das Ubuntu-Paketverwaltungssystem eingebunden. Dadurch kann der Paketmanager Terraform direkt aus dem offiziellen Repository beziehen und zukünftig aktualisieren.

Nach der Einbindung des Repositories wurden die Paketlisten mit `sudo apt update` aktualisiert. Dadurch erkannte der Paketmanager die neu hinzugefügte Paketquelle und stellte die dort verfügbaren Terraform-Pakete für die Installation bereit.

```

alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp) $ sudo apt update
Get:1 https://cli.github.com/packages stable InRelease [3,917 B]
Get:2 https://apt.releases.hashicorp.com noble InRelease [12.9 kB]
Get:3 https://packages.cloud.google.com/apt gcsfuse-noble InRelease [1,227 B]
Get:4 https://packages.cloud.google.com/apt cloud-sdk InRelease [1,620 B]
Get:5 https://cli.github.com/packages stable/main amd64 Packages [354 B]
Get:6 https://apt.releases.hashicorp.com noble/main amd64 Packages [298 kB]
Get:7 https://packages.cloud.google.com/apt gcsfuse-noble/main amd64 Packages [73.6 kB]

```

Abbildung 18: Aktualisierung der Paketlisten nach Einbindung des HashiCorp-Repositories.

Anschließend wurde Terraform mithilfe des Ubuntu-Paketmanagers installiert. Dabei wurden sämtliche für den Betrieb erforderlichen Programmdateien automatisch eingerichtet und in das System integriert.

```

alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp) $ sudo apt install terraform -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  terraform

```

Abbildung 19: Installation von Terraform über den Paketmanager

Nach Abschluss der Installation erfolgte die Überprüfung der installierten Terraform-Version. Zusätzlich erfolgte die Ausgabe über das Binary `/usr/bin/terraform version`, um zu bestätigen, dass die zuvor installierte Version erfolgreich eingebunden wurde. Damit wurde die erfolgreiche Installation von Terraform nachgewiesen [9].

```

alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp) $ /usr/bin/terraform version
Terraform v1.15.7
on linux_amd64
+ provider registry.terraform.io/hashicorp/google v5.45.2

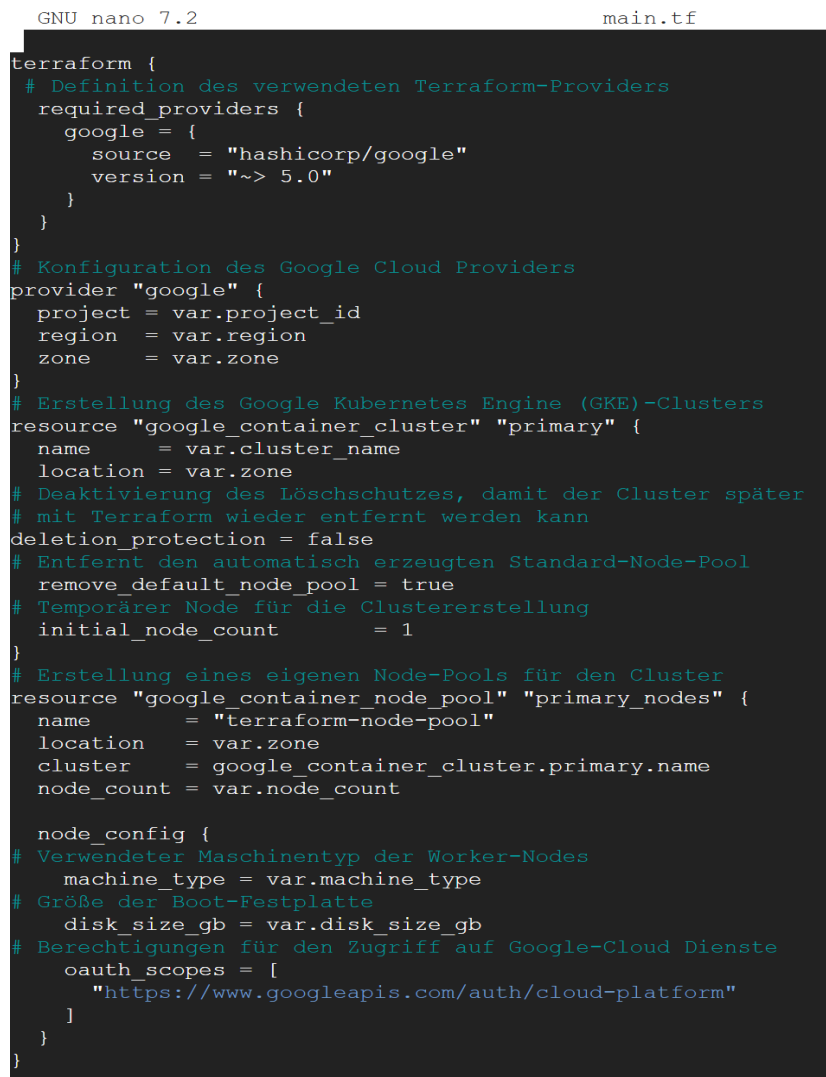
```

Abbildung 20: Überprüfung der installierten Terraform-Version

6.3 Aufbau der Terraform-Konfigurationsdateien

Die Terraform-Konfiguration wurde in mehrere Dateien aufgeteilt, um die Infrastruktur übersichtlich und wartbar zu gestalten. Verwendet wurden die Dateien `main.tf`, `variables.tf` und `outputs.tf`. Diese Aufteilung verbessert die Struktur der Konfiguration und erleichtert spätere Anpassungen einzelner Infrastrukturparameter.

Die Datei `main.tf` bildet den zentralen Bestandteil der Terraform-Konfiguration. In ihr werden sowohl der Google-Provider als auch die für die automatische Bereitstellung des GKE-Clusters und des zugehörigen Node-Pools benötigten Ressourcen definiert. Sämtliche Infrastrukturkomponenten werden in dieser Datei definiert und anschließend von Terraform automatisch bereitgestellt [3].

The image shows a screenshot of a GNU nano 7.2 text editor window. The title bar at the top indicates the editor is 'GNU nano 7.2' and the file being edited is 'main.tf'. The editor's background is dark, and the text is light-colored. The code is written in Terraform syntax and includes several comments in German. It defines the required providers (Google), configures the Google Cloud provider with variables for project, region, and zone, creates a Google Kubernetes Engine (GKE) cluster named 'primary', and creates a separate node pool named 'primary_nodes'. The node pool configuration includes details like machine type, disk size, and OAuth scopes. The code uses variables like 'var.project_id', 'var.region', 'var.zone', 'var.cluster_name', 'var.machine_type', 'var.disk_size_gb', and 'var.node_count' for flexibility.

```
GNU nano 7.2 main.tf

terraform {
  # Definition des verwendeten Terraform-Providers
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 5.0"
    }
  }
}

# Konfiguration des Google Cloud Providers
provider "google" {
  project = var.project_id
  region  = var.region
  zone    = var.zone
}

# Erstellung des Google Kubernetes Engine (GKE)-Clusters
resource "google_container_cluster" "primary" {
  name     = var.cluster_name
  location = var.zone
  # Deaktivierung des Löschschutzes, damit der Cluster später
  # mit Terraform wieder entfernt werden kann
  deletion_protection = false
  # Entfernt den automatisch erzeugten Standard-Node-Pool
  remove_default_node_pool = true
  # Temporärer Node für die Clustererstellung
  initial_node_count       = 1
}

# Erstellung eines eigenen Node-Pools für den Cluster
resource "google_container_node_pool" "primary_nodes" {
  name        = "terraform-node-pool"
  location    = var.zone
  cluster     = google_container_cluster.primary.name
  node_count  = var.node_count

  node_config {
    # Verwendeter Maschinentyp der Worker-Nodes
    machine_type = var.machine_type
    # Größe der Boot-Festplatte
    disk_size_gb = var.disk_size_gb
    # Berechtigungen für den Zugriff auf Google-Cloud Dienste
    oauth_scopes = [
      "https://www.googleapis.com/auth/cloud-platform"
    ]
  }
}
```

Abbildung 21: Terraform-Konfiguration in der Datei `main.tf`

Abbildung 21 zeigt die in dieser Projektarbeit verwendete `main.tf`. Erkennbar sind die Definition des Google-Providers, die Konfiguration des GKE-Clusters sowie die Erstellung eines separaten Node-Pools einschließlich der benötigten Infrastrukturparameter.

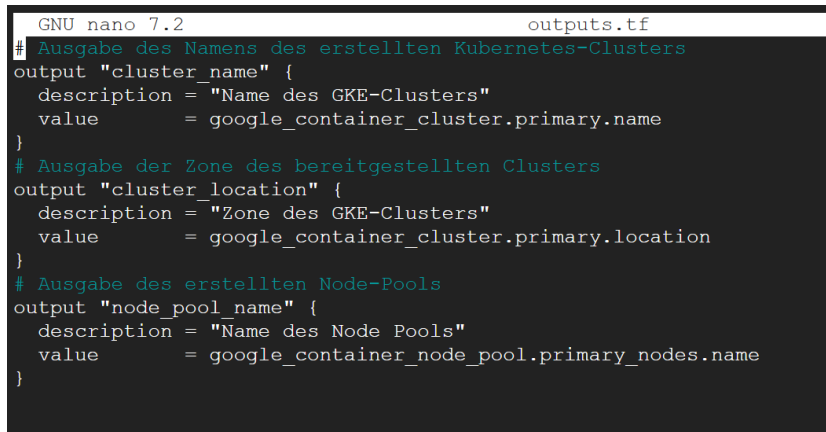
Um projektspezifische Einstellungen nicht mehrfach innerhalb der Konfiguration hinterlegen zu müssen, wurden zentrale Parameter in die Datei `variables.tf` ausgelagert.

```
GNU nano 7.2                                     variables.tf
# Projektparameter der Google Cloud Platform
variable "project_id" {
  description = "Google Cloud Projekt-ID"
  type        = string
  default     = "pa-kubernetes-terraform-gcp"
}
# Region der Bereitstellung
variable "region" {
  description = "Google Cloud Region"
  type        = string
  default     = "europe-west3"
}
# Zone für den Kubernetes-Cluster
variable "zone" {
  description = "Google Cloud Zone"
  type        = string
  default     = "europe-west3-b"
}
# Anzahl der Worker-Nodes
variable "node_count" {
  description = "Anzahl der Worker-Nodes"
  type        = number
  default     = 1
}
# Größe der Boot-Festplatte pro Node
variable "disk_size_gb" {
  description = "Größe des Bootlaufwerks je Node in GB"
  type        = number
  default     = 20
}
# Name des Kubernetes-Clusters
variable "cluster_name" {
  description = "Name des GKE-Clusters"
  type        = string
  default     = "terraform-gke-cluster"
}
# Maschinentyp der Worker-Nodes
variable "machine_type" {
  description = "Maschinentyp der Worker-Nodes"
  type        = string
  default     = "e2-small"
}
```

Abbildung 22: Definition der Terraform-Variablen

Abbildung 22 zeigt die Definition der verwendeten Variablen. Hierzu gehören unter anderem die Projekt-ID, Region, Zone, Clusternamen, Anzahl der Worker-Nodes, der Maschinentyp sowie die Größe des Bootlaufwerks. Dadurch bleibt die Infrastrukturdefinition in der Datei `main.tf` übersichtlich und kann bei Änderungen zentral angepasst werden.

Nach der erfolgreichen Bereitstellung der Infrastruktur sollten wichtige Informationen automatisch ausgegeben werden. Hierfür wurde die Datei `outputs.tf` verwendet.



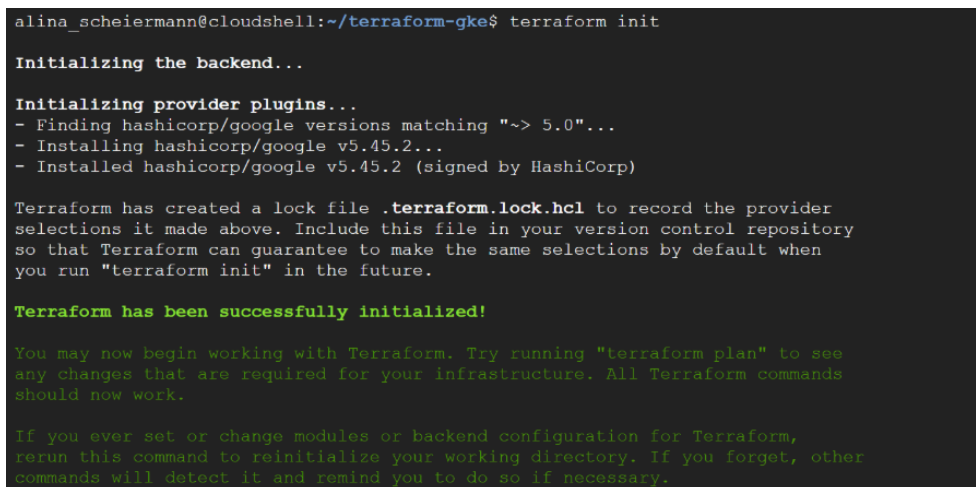
```
GNU nano 7.2 outputs.tf
# Ausgabe des Namens des erstellten Kubernetes-Clusters
output "cluster_name" {
  description = "Name des GKE-Clusters"
  value       = google_container_cluster.primary.name
}
# Ausgabe der Zone des bereitgestellten Clusters
output "cluster_location" {
  description = "Zone des GKE-Clusters"
  value       = google_container_cluster.primary.location
}
# Ausgabe des erstellten Node-Pools
output "node_pool_name" {
  description = "Name des Node Pools"
  value       = google_container_node_pool.primary_nodes.name
}
```

Abbildung 23: Definition der Terraform-Ausgabewerte

Abbildung 23 zeigt die Definition der Ausgabewerte. Terraform liefert nach erfolgreicher Ausführung unter anderem den Namen des Clusters, dessen Standort sowie den Namen des erzeugten Node-Pools aus. Dadurch können wichtige Eigenschaften der bereitgestellten Infrastruktur unmittelbar überprüft werden [3].

6.4 Initialisierung und Validierung der Terraform-Konfiguration

Nachdem die Terraform-Konfigurationsdateien (`main.tf`, `variables.tf` und `outputs.tf`) erstellt worden waren, wurde das Arbeitsverzeichnis mit dem Befehl `terraform init` initialisiert. Dabei lädt Terraform automatisch die in der Konfiguration definierten Provider herunter und richtet die Arbeitsumgebung für die weitere Verarbeitung der Konfigurationsdateien ein. Erst nach erfolgreicher Initialisierung können weitere Terraform-Befehle wie `terraform validate`, `terraform plan` oder `terraform apply` ausgeführt werden [11].



```
alina_scheiermann@cloudshell:~/terraform-gke$ terraform init

Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/google versions matching ">= 5.0"...
- Installing hashicorp/google v5.45.2...
- Installed hashicorp/google v5.45.2 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Abbildung 24: Initialisierung der Terraform-Konfiguration

Nach der erfolgreichen Initialisierung wurde die Konfiguration mit dem Befehl `terraform validate` überprüft. Dabei kontrolliert Terraform die Syntax sowie die logische Struktur der Konfigurationsdateien, ohne Änderungen an der Cloud-Infrastruktur vorzunehmen. Die erfolgreiche Validierung zeigt, dass die Konfiguration fehlerfrei ist und für die weitere Bereitstellung verwendet werden kann [11].

```
alina_scheiermann@cloudshell:~/terraform-gke$ terraform validate
Success! The configuration is valid.
alina_scheiermann@cloudshell:~/terraform-gke$
```

Abbildung 25: Validierung der Terraform-Konfiguration

6.5 Terraform Plan

Nach der erfolgreichen Validierung wurde mit dem Befehl `terraform plan` eine Vorschau der geplanten Infrastruktur erstellt. Die Ausgabe zeigt die Erstellung eines Google Kubernetes Engine (GKE)-Clusters sowie eines zugehörigen Node-Pools. Insgesamt sollten zwei Ressourcen erzeugt werden, während keine bestehenden Ressourcen geändert oder gelöscht werden. Dadurch ließ sich die geplante Infrastruktur vor der eigentlichen Bereitstellung überprüfen. [12].

```
alina_scheiermann@cloudshell:~/terraform-gke$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

# google_container_cluster.primary will be created
+ resource "google_container_cluster" "primary" {
  + cluster_ipv4_cidr           = (known after apply)
  + datapath_provider           = (known after apply)
  + default_max_pods_per_node   = (known after apply)
  + deletion_protection         = true
  + enable_cilium_clusterwide_network_policy = false
  + enable_intranode_visibility = (known after apply)
  + enable_kubernetes_alpha     = false
  + enable_l4_ilb_subsetting    = false
  + enable_legacy_abac          = false
  + enable_multi_networking      = false
  + enable_shielded_nodes       = true
  + endpoint                    = (known after apply)
  + id                          = (known after apply)
  + initial_node_count          = 1
  + label_fingerprint           = (known after apply)
  + location                    = "europe-west3-b"
  + logging_service              = (known after apply)
  + master_version               = (known after apply)
  + monitoring_service           = (known after apply)
  + name                         = "terraform-gke-cluster"
  + network                     = "default"
  + networking_mode              = (known after apply)
  + node_locations               = (known after apply)
  + node_version                 = (known after apply)
  + operation                    = (known after apply)
  + private_ipv6_google_access   = (known after apply)
  + project                     = (known after apply)
  + remove_default_node_pool     = true
  + self_link                   = (known after apply)
  + services_ipv4_cidr           = (known after apply)
  + subnetwork                   = (known after apply)
  + tpu_ipv4_cidr_block          = (known after apply)
}

# google_container_node_pool.primary_nodes will be created
+ resource "google_container_node_pool" "primary_nodes" {
  + cluster = "terraform-gke-cluster"
  + id      = (known after apply)
}

Plan: 2 to add, 0 to change, 0 to destroy.
```

Abbildung 26: Terraform-Plan für die Bereitstellung des GKE-Clusters

6.6 Terraform Apply

Nach der Überprüfung des Ausführungsplans wurde die Infrastruktur mit `terraform apply` bereitgestellt. Terraform erzeugte automatisch den GKE-Cluster sowie den zugehörigen Node-Pool und bestätigte die erfolgreiche Erstellung aller definierten Ressourcen (Abbildung 27) [13].

```
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:

cluster_location = "europe-west3-b"
cluster_name     = "terraform-gke-cluster"
node_pool_name   = "terraform-node-pool"
alina_scheiermann@cloudshell:~/terraform-gke$
```

Abbildung 27: Bereitstellung des GKE-Clusters mit Terraform

Die erfolgreiche Bereitstellung konnte zusätzlich in der Google Cloud Console überprüft werden. Der neu erzeugte Cluster wurde dort als aktive Kubernetes-Ressource angezeigt (Abbildung 28).

6.7 Überprüfung des GKE-Clusters

Nach erfolgreicher Ausführung des Befehls `terraform apply` wurde überprüft, ob der Kubernetes-Cluster erfolgreich in der Google Cloud Platform bereitgestellt wurde. Hierzu erfolgte zunächst eine Kontrolle in der Google Cloud Console. Dabei wurde überprüft, ob der Cluster erfolgreich bereitgestellt wurde und den in der Terraform-Konfiguration definierten Parametern entspricht [13].

Zustand ?

100 % fehlerfrei

Keine Empfehlungen

Upgrade ?

100 % auf dem neuesten Stand

Keine Empfehlungen

Geschätzte Kosten pro Monat ?

0,00 \$/Monat · 0%

Keine Empfehlungen

Filter

Name oder Wert des Attributs eingeben

?

☐	Status	Name ↑	Speicherort	Anzahl der Knoten	vCPUs insgesamt	Speicher
☒		terraform-gke-cluster	europe-west3-b	1	2	

Abbildung 28: Mit Terraform bereitgestellter GKE-Cluster in der Google Cloud Console

Die Detailansicht des Clusters bestätigt den betriebsbereiten Zustand der bereitgestellten Infrastruktur. Der Cluster befand sich im Status Running und verfügte über einen Worker-Node. Damit konnte die erfolgreiche Bereitstellung nachgewiesen werden. (Abbildung 29).

Die Eigenschaften des bereitgestellten Node-Pools entsprechen den in der Terraform-Konfiguration definierten Parametern. Der Node-Pool wurde mit einem Worker-Node des Typs e2-small erstellt. Zusätzlich wurde ein Bootlaufwerk mit einer Kapazität von 20 GB bereitgestellt.

✓ terraform-gke-cluster

Übersicht	Details	Knoten	Speicher	Beobachtbarkeit	Logs	Anwendu
Status Running	Modus Standard	Anzahl der Knoten 1	Zone der Steuerungsebene europe-west3-b	Endpoint 35.198.87.80		

Abbildung 29: Detailansicht des mit Terraform bereitgestellten GKE-Clusters

Zur vollständigen Überprüfung der bereitgestellten Infrastruktur wurde anschließend der automatisch erzeugte Node-Pool kontrolliert. Dabei wurde kontrolliert, ob die in der Terraform-Konfiguration definierten Parameter, insbesondere der Maschinentyp und die Anzahl der Worker-Nodes, korrekt übernommen wurden.

Name ↑	Status	Version	Knotenpooltyp ?	Anzahl der Knoten	Computertyp
terraform-node-pool	✓ Ok	1.35.5-gke.1000000	Vom Nutzer verwaltet	1	e2-small

Abbildung 30: Durch Terraform erzeugter Node-Pool mit einem Worker-Node vom Typ e2-small

6.8 Verbindung zum Cluster

Nach der Bereitstellung des Clusters wurde mit `gcloud container clusters get-credentials` eine Verbindung zum Kubernetes-Cluster hergestellt. Anschließend wurde mit `kubectl get nodes` überprüft, ob der bereitgestellte Worker-Node verfügbar ist. Die Ausgabe zeigt, dass der Node erfolgreich bereitgestellt wurde und sich im Status *Ready* befindet [15].

```
alina_scheiermann@cloudshell:~$ gcloud config set project pa-kubernetes-terraform-gcp
Updated property [core/project].
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp)$ gcloud container clusters get-credentials terraform-gke-cluster --zone europe-west3-b
Fetching cluster endpoint and auth data.
kubeconfig entry generated for terraform-gke-cluster.
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp)$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
gke-terraform-gke-cl-terraform-node-p-5172dbc7-6j7s  Ready    <none>   50m   v1.35.5-gke.1000000
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp)$
```

Abbildung 31: Verbindung zum GKE-Cluster und Überprüfung des Worker-Nodes

6.9 Bereitstellung der Testanwendung

Zur Überprüfung der Funktionsfähigkeit des bereitgestellten GKE-Clusters wurde eine NGINX-Anwendung als Testanwendung erstellt. Hierzu wurde ein Deployment auf Basis des offiziellen NGINX-Container-Images angelegt. Die anschließende Überprüfung bestätigte die erfolgreiche Erstellung des Deployments (Abbildung 32) [2].

```
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp) $ kubectl create deployment nginx --image=nginx
deployment.apps/nginx created
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp) $ kubectl get deployments
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
nginx	1/1	1	1	17s

Abbildung 32: Erstellung eines NGINX-Deployments auf dem per Terraform bereitgestellten GKE-Cluster

6.10 Skalierung der Anwendung

Zur Demonstration der Skalierbarkeit von Kubernetes wurde die Anzahl der Replikate des NGINX-Deployments von einem auf drei erhöht. Kubernetes erzeugte daraufhin automatisch zwei zusätzliche Pods. Die Überprüfung mit `kubectl get pods` bestätigte, dass alle drei Pods erfolgreich gestartet wurden und sich im Status *Running* befinden (Abbildung 33) [2].

```
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp) $ kubectl scale deployment nginx --replicas=3
deployment.apps/nginx scaled
alina_scheiermann@cloudshell:~ (pa-kubernetes-terraform-gcp) $ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
nginx-56c45fd5ff-6qxdx	1/1	Running	0	11s
nginx-56c45fd5ff-9gtwt	1/1	Running	0	114s
nginx-56c45fd5ff-h8dtt	1/1	Running	0	11s

Abbildung 33: Skalierung des NGINX-Deployments von einem auf drei Pods im GKE-Cluster

6.11 Skalierung des Kubernetes-Clusters

Zur Demonstration der Skalierung des Kubernetes-Clusters wurde die Anzahl der Worker-Nodes im Terraform-Node-Pool von einem auf drei erhöht. Hierzu wurde der Parameter `node_count` in `variables.tf` innerhalb der Terraform-Konfiguration angepasst und die Änderungen anschließend mit `terraform apply` übernommen. Nach erfolgreicher Bereitstellung bestätigte die Ausgabe des Befehls `kubectl get nodes`, dass drei Worker-Nodes im Status *Ready* verfügbar waren. Damit konnte die Skalierung der Cluster-Infrastruktur erfolgreich nachgewiesen werden [3].

```
alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp) $ gcloud container clusters get-credentials terraform-gke-cluster --zone europe-west3-b
Fetching cluster endpoint and auth data.
kubeconfig entry generated for terraform-gke-cluster.
alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp) $ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
gke-terraform-gke-cl-terraform-node-p-8a2ff290-c979e.1057002	Ready	<none>	2m17s	v1.35.5-gk
gke-terraform-gke-cl-terraform-node-p-8a2ff290-kzfb e.1057002	Ready	<none>	2m14s	v1.35.5-gk
gke-terraform-gke-cl-terraform-node-p-8a2ff290-lq29 e.1057002	Ready	<none>	2m15s	v1.35.5-gk

```
alina_scheiermann@cloudshell:~/terraform-gke (pa-kubernetes-terraform-gcp) $
```

Abbildung 34: Skalierter GKE-Cluster mit drei Worker-Nodes

6.12 Rückbau der Infrastruktur

Nach erfolgreicher Bereitstellung und Überprüfung des Kubernetes-Clusters wurde die mit Terraform erstellte Cloud-Infrastruktur wieder entfernt. Hierfür wurde der Befehl `terraform destroy` verwendet, der sämtliche durch Terraform verwalteten Ressourcen kontrolliert löscht. Dieser Schritt dient dazu, nicht mehr benötigte Cloud-Ressourcen freizugeben und unnötige Kosten zu vermeiden [14].

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes
```

Abbildung 35: Bestätigung der Löschung der Terraform-Infrastruktur

Nach Bestätigung der Löschung begann Terraform mit dem Rückbau der zuvor bereitgestellten Infrastruktur. Dabei wurden sowohl der GKE-Cluster als auch der zugehörige Node-Pool vollständig entfernt. Die erfolgreiche Ausführung bestätigte, dass sämtliche durch Terraform verwalteten Ressourcen gelöscht wurden.

```
lapsed]
google_container_cluster.primary: Still destroying... [id=projects/pa-kube
netes-terraform-gcp/10...west3-b/clusters/terraform-gke-cluster, 04m10s e
lapsed]
google_container_cluster.primary: Destruction complete after 4m12s

Destroy complete! Resources: 2 destroyed.
```

Abbildung 36: Rückbau der mit Terraform bereitgestellten Infrastruktur

Nach erfolgreicher Ausführung des Befehls `terraform destroy` wurde die Löschung zusätzlich in der Google Cloud Console überprüft. Dabei konnte festgestellt werden, dass der zuvor erstellte Kubernetes-Cluster nicht mehr vorhanden war. Dadurch konnte nachgewiesen werden, dass sämtliche durch Terraform bereitgestellten Ressourcen erfolgreich entfernt wurden und keine unnötigen Cloud-Ressourcen oder zusätzlichen Kosten mehr verursachen [14].

Nach Abschluss aller praktischen Experimente standen noch 40,55 US-Dollar (81 %) des ursprünglich bereitgestellten Google-Cloud-Guthabens zur Verfügung. Dies bestätigt, dass sämtliche Versuche innerhalb des bereitgestellten Budgets durchgeführt wurden und durch den konsequenten Rückbau der Cloud-Ressourcen keine unnötigen Kosten entstanden [8].

Guthabenname	Status ↑	Prozent verbleibend	Restwert	Ursprünglicher Wert	Typ	ID des Guthabens
Computer & Media Networking II	✓ Verfügbar	81 %	40,55 \$	50,00 \$	Einmal	GP4XHY2LJBD4Y5Y3

Abbildung 37 Verbleibendes Google-Cloud-Guthaben nach Abschluss der Projektarbeit

7 Vergleich der manuellen Bereitstellung mit der automatisierten Bereitstellung durch Terraform

Im Rahmen dieser Projektarbeit wurden zwei unterschiedliche Verfahren zur Einrichtung einer Kubernetes-Umgebung praktisch umgesetzt und miteinander verglichen. Hierfür erfolgte zunächst die manuelle Installation einer Kubernetes-Umgebung mit MicroK8s auf einer virtuellen Maschine der Google Compute Engine. Anschließend wurde ein Kubernetes-Cluster in der Google Kubernetes Engine (GKE) mithilfe von Terraform automatisiert bereitgestellt.

Die manuelle Installation vermittelte einen tiefen Einblick in den Aufbau und die Funktionsweise eines Kubernetes-Clusters. Sämtliche Installations- und Konfigurationsschritte wurden selbst durchgeführt, wodurch sich die einzelnen Komponenten sowie mögliche Fehlerquellen besser nachvollziehen ließen. Dieses Verfahren eignet sich insbesondere zum Erlernen der Kubernetes-Architektur und zum Verständnis der zugrunde liegenden Systemprozesse.

Die automatisierte Bereitstellung mit Terraform folgte dem Infrastructure-as-Code-Prinzip. Sämtliche Ressourcen wurden deklarativ in Konfigurationsdateien beschrieben und konnten dadurch reproduzierbar erstellt sowie bei Bedarf angepasst oder erneut bereitgestellt werden. Dadurch verringerte sich der manuelle Aufwand erheblich und Änderungen an der Infrastruktur ließen sich zentral über die Terraform-Konfiguration verwalten.

Der praktische Vergleich zeigte, dass beide Verfahren unterschiedliche Stärken besitzen. Während die manuelle Installation vor allem das Verständnis der zugrunde liegenden Kubernetes-Komponenten fördert, ermöglicht Terraform eine schnelle, standardisierte und reproduzierbare Bereitstellung der benötigten Cloud-Infrastruktur. Die im Rahmen der Projektarbeit durchgeführten Demonstrationen der Skalierung und der Self-Healing-Funktion verdeutlichten darüber hinaus die Stärken von Kubernetes als Container-Orchestrierungsplattform. Während die Skalierung eine flexible Anpassung der Anzahl laufender Pods ermöglichte, zeigte die Self-Healing-Demonstration, dass Kubernetes den definierten Sollzustand eines Deployments automatisch wiederherstellt und dadurch die Verfügbarkeit der Anwendung auch nach dem Ausfall eines Pods sicherstellt. Insbesondere bei wiederholten Bereitstellungen oder produktiven Cloud-Umgebungen bietet die Kombination aus Kubernetes und Terraform daher deutliche Vorteile hinsichtlich Automatisierung, Reproduzierbarkeit und Ausfallsicherheit [2–4].

Tabelle 1 fasst die wesentlichen Unterschiede zwischen der manuellen Bereitstellung mit MicroK8s und der automatisierten Bereitstellung mit Terraform zusammen.

Manuelle Installation (MicroK8s)	Terraform (GKE)
Schrittweise Installation	Automatische Bereitstellung
Direkte Konfiguration der Kubernetes-Umgebung	Infrastruktur über Konfigurationsdateien
Höherer manueller Aufwand	Geringerer manueller Aufwand
Installation auf einer Compute-Engine-VM	Bereitstellung eines GKE-Clusters
Änderungen müssen manuell durchgeführt werden	Änderungen erfolgen über Anpassungen der Terraform-Dateien
Gut geeignet zum Erlernen der Kubernetes-Architektur	Gut geeignet für reproduzierbare Bereitstellung
Erneute Bereitstellung durch wiederholte Ausführung der Installationsschritte	Über Terraform erneut ausführbar

Tabelle 1: Vergleich MicroK8s & Terraform

Die Gegenüberstellung verdeutlicht, dass sich beide Verfahren sinnvoll ergänzen. Die manuelle Installation eignet sich insbesondere für Lern- und Demonstrationszwecke, da sie ein detailliertes Verständnis der Kubernetes-Architektur vermittelt. Terraform ermöglicht dagegen eine reproduzierbare und effizient verwaltbare Bereitstellung der Cloud-Infrastruktur. In Kombination mit den in dieser Projektarbeit demonstrierten Kubernetes-Funktionen wie Skalierung und Self-Healing entsteht eine leistungsfähige Grundlage für den zuverlässigen Betrieb containerisierter Anwendungen in Cloud-Umgebungen.

8 Fazit

Im Rahmen dieser Projektarbeit wurden die in der Aufgabenstellung geforderten Experimente zur manuellen und automatisierten Bereitstellung einer Kubernetes-Umgebung in der Google Cloud Platform erfolgreich durchgeführt und dokumentiert. Hierzu wurde zunächst eine Kubernetes-Umgebung mithilfe von MicroK8s auf einer Instanz der Google Compute Engine manuell eingerichtet. Anschließend erfolgte die automatisierte Bereitstellung eines Kubernetes-Clusters in der Google Kubernetes Engine (GKE) mithilfe von Terraform. Die Funktionsfähigkeit beider Umgebungen wurde durch die Bereitstellung einer NGINX-Testanwendung überprüft. Darüber hinaus wurden die Skalierung einer Anwendung sowie die Self-Healing-Funktion von Kubernetes erfolgreich demonstriert.

Die praktische Durchführung hat gezeigt, dass beide Bereitstellungsverfahren unterschiedliche Stärken und Einsatzschwerpunkte besitzen. Die manuelle Installation mit MicroK8s ermöglichte einen direkten Einblick in die Konfiguration und Funktionsweise einer Kubernetes-Umgebung und erleichterte das Verständnis der zugrunde liegenden Komponenten. Die automatisierte Bereitstellung mit Terraform verdeutlichte dagegen die Vorteile des Infrastructure-as-Code-Ansatzes. Durch die Verwendung deklarativer Konfigurationsdateien konnten Cloud-Ressourcen reproduzierbar, nachvollziehbar und mit

geringem manuellem Aufwand bereitgestellt, angepasst und nach Abschluss der Experimente wieder entfernt werden.

Besonders deutlich wurden die Stärken von Kubernetes anhand der praktischen Demonstrationen. Die Skalierung zeigte, dass Anwendungen durch die Anpassung der Replikanzzahl flexibel an veränderte Anforderungen angepasst werden können. Die Demonstration der Self-Healing-Funktion verdeutlichte darüber hinaus eine der zentralen Eigenschaften einer Container-Orchestrierungsplattform. Durch das gezielte Löschen eines Pods konnte nachgewiesen werden, dass Kubernetes nicht einzelne Pods überwacht, sondern den im Deployment definierten Sollzustand automatisch wiederherstellt. Der ausgefallene Pod wurde ohne manuellen Eingriff ersetzt, sodass die gewünschte Anzahl an Replikaten erhalten blieb und die Anwendung weiterhin verfügbar war. Dieses Verhalten verdeutlicht eine wesentliche Aufgabe von Kubernetes und zeigt, wie durch die automatische Wiederherstellung ausgefallener Pods die Verfügbarkeit containerisierter Anwendungen sichergestellt werden kann.

Nach Abschluss der praktischen Experimente wurden die verwendeten Cloud-Ressourcen ordnungsgemäß beendet beziehungsweise mit `terraform destroy` vollständig entfernt. Dadurch konnten unnötige Cloud-Kosten vermieden werden. Das bereitgestellte Google-Cloud-Guthaben wurde nur zu einem geringen Teil genutzt, sodass sämtliche Experimente innerhalb des verfügbaren Budgets durchgeführt werden konnten.

Insgesamt konnten die wesentlichen Ziele der Projektarbeit erfolgreich erreicht werden. Die Ergebnisse der Projektarbeit zeigen, dass sich die manuelle Bereitstellung mit MicroK8s und die automatisierte Bereitstellung mit Terraform sinnvoll ergänzen. Während die manuelle Installation insbesondere zum Verständnis der Kubernetes-Architektur und ihrer Funktionsweise beiträgt, ermöglicht Terraform eine standardisierte und reproduzierbare Bereitstellung der benötigten Cloud-Infrastruktur. Die entstandene Dokumentation der durchgeführten Experimente und Konfigurationsschritte kann darüber hinaus als Grundlage für zukünftige Projekte im Bereich Kubernetes, Terraform und Cloud Computing dienen.

Quellenverzeichnis

- [1] Docker Inc., „What is a Container?“, [Online]. verfügbar unter:
<https://www.docker.com/resources/what-container/> , Zugriff am 21.06.2026.
- [2] Kubernetes Authors, *Kubernetes Documentation – Concepts Overview*, verfügbar unter:
<https://kubernetes.io/docs/concepts/overview/> , Zugriff am 21.06.2026.
- [3] HashiCorp, *Terraform Documentation*, verfügbar unter:
<https://developer.hashicorp.com/terraform/docs> , Zugriff am 21.06.2026.
- [4] Canonical Ltd., *MicroK8s Documentation*, verfügbar unter: <https://microk8s.io/docs> ,
Zugriff am 21.06.2026.
- [5] Google Cloud, *Compute Engine Documentation*, verfügbar unter:
<https://cloud.google.com/compute/docs> , Zugriff am 18.06.2026.
- [6] Google Cloud, *Google Kubernetes Engine (GKE) Documentation*, verfügbar unter:
<https://cloud.google.com/kubernetes-engine/docs> , Zugriff am 20.06.2026.
- [7] Google Cloud, *Cloud Shell Documentation*, verfügbar unter:
<https://cloud.google.com/shell/docs> , Zugriff am 19.06.2026.
- [8] Google Cloud, *Cloud Billing – Pricing Overview*, verfügbar unter:
<https://cloud.google.com/billing/docs/how-to/pricing-overview> , Zugriff am 24.06.2026.
- [9] HashiCorp, *Install Terraform*, verfügbar unter:
<https://developer.hashicorp.com/terraform/install> , Zugriff am 22.06.2026.
- [10] HashiCorp, *Google Provider Documentation*, verfügbar unter:
<https://registry.terraform.io/providers/hashicorp/google/latest/docs> , Zugriff am 23.06.2026.
- [11] HashiCorp, *terraform validate*, verfügbar unter:
<https://developer.hashicorp.com/terraform/cli/commands/validate> , Zugriff am 22.06.2026.
- [12] HashiCorp, *terraform plan*, verfügbar unter:
<https://developer.hashicorp.com/terraform/cli/commands/plan> , Zugriff am 23.06.2026.
- [13] HashiCorp, *terraform apply*, verfügbar unter:
<https://developer.hashicorp.com/terraform/cli/commands/apply> , Zugriff am 24.06.2026.
- [14] HashiCorp, *terraform destroy*, verfügbar unter:
<https://developer.hashicorp.com/terraform/cli/commands/destroy> , Zugriff am 25.06.2026.
- [15] Google Cloud, *Google Cloud CLI Documentation*, verfügbar unter:
<https://cloud.google.com/sdk/gcloud> , Zugriff am 19.06.2026.

Abbildungsverzeichnis

Abbildung 1: Bereitgestelltes Google-Cloud-Guthaben zu Beginn der Projektarbeit	7
Abbildung 2: Konfiguration der Compute-Engine-Instanz	12
Abbildung 3: Erstellte VM-Instanz	12
Abbildung 4: SSH-Verbindung zur Ubuntu-VM	13
Abbildung 5: Überprüfung der Snap-Installation	14
Abbildung 6: Installation von MicroK8s	14
Abbildung 7: Fehlende Benutzerberechtigungen	14
Abbildung 8: Initialisierung von MicroK8s	15
Abbildung 9: Kubernetes Node nach erfolgreicher Initialisierung	15
Abbildung 10: Erstellung NGINX Deployments	15
Abbildung 11: Skalierung auf drei Pods	16
Abbildung 12: NGINX-Service nach erfolgreicher Skalierung	16
Abbildung 13: Zugriff auf die NGINX-Anwendung	16
Abbildung 14: Ausgangszustand des NGINX-Deployments mit drei laufenden Pods vor der Demonstration der Self-Healing-Funktion.	17
Abbildung 15: Manuelles Löschen eines Pods und automatische Wiederherstellung des Deployments durch Kubernetes (Self-Healing).	17
Abbildung 16: Kontrolliertes Stoppen des MicroK8s-Clusters	18
Abbildung 17: Import des offiziellen HashiCorp-GPG-Schlüssels	19
Abbildung 18: Aktualisierung der Paketlisten nach Einbindung des HashiCorp-Repositories	20
Abbildung 19: Installation von Terraform über den Paketmanager apt	20
Abbildung 20: Überprüfung der installierten Terraform-Version	20
Abbildung 21: Terraform-Konfiguration in der Datei main.tf	21
Abbildung 22: Definition der Terraform-Variablen	22
Abbildung 23: Definition der Terraform-Ausgabewert	23
Abbildung 24: Initialisierung der Terraform-Konfiguration	23
Abbildung 25: Validierung der Terraform-Konfiguration	24

Abbildung 26: Terraform-Plan für die Bereitstellung des GKE-Clusters	24
Abbildung 27: Bereitstellung des GKE-Clusters mit Terraform	25
Abbildung 28: Mit Terraform bereitgestellter GKE-Cluster in der Google Cloud Console	25
Abbildung 29: Detailansicht des mit Terraform bereitgestellten GKE-Clusters	26
Abbildung 30: Durch Terraform erzeugter Node-Pool mit einem Worker-Node vom Typ e2-small	26
Abbildung 31: Verbindung zum GKE-Cluster und Überprüfung des Worker-Nodes	26
Abbildung 32: Erstellung eines NGINX-Deployments auf dem per Terraform bereitgestellten GKE-Cluster	27
Abbildung 33: Skalierung des NGINX-Deployments von einem auf drei Pods im GKE-Cluster	27
Abbildung 34: Skalierter GKE-Cluster mit drei Worker-Nodes	27
Abbildung 35: Bestätigung der Löschung der Terraform-Infrastruktur	28
Abbildung 36: Rückbau der mit Terraform bereitgestellten Infrastruktur	28
Abbildung 37: Verbleibendes Google-Cloud-Guthaben nach Abschluss der Projektarbeit.	28

Tabellenverzeichnis

Tabelle 1: Vergleich MicroK8s & Terraform	30
---	----